automation

MDPI

Article

Real-Time Vehicle Sticker Recognition for Smart Gate Control with YOLOv8 and Raspberry Pi 4

Serosh Karim Noon [1,*], Ali Hassan Noor [1], Abdul Mannan [2,*], Miqdam Arshad [1], Turab Haider [1] and Muhammad Abdullah [1]

[1] Department of Electrical Engineering, NFC Institute of Engineering & Technology, Multan 60000, Pakistan; engr.alihassannoor@gmail.com (A.H.N.); miqdamarshad4488@gmail.com (M.A.); engrturab12@gmail.com (T.H.); raomabdullah69@gmail.com (M.A.)
[2] Department of Biomedical Engineering, NFC Institute of Engineering & Technology, Multan 60000, Pakistan
* Correspondence: seroshkarim@nfciet.edu.pk (S.K.N.); mannan@nfciet.edu.pk (A.M.)

Abstract

In today's fast-paced world, secure and efficient access control is crucial for places like schools, gated communities, and corporate campuses. The system must overcome the issues of manual checking and record maintenance of traditional methods like RFID cards or license plate recognition. Our work introduces a budget-friendly, automated solution. A prototype was developed for a vehicle sticker recognition system to control and monitor gate access at NFC IET University as a case study. The automated system design will replace manual checking by detecting the car stickers issued to each vehicle by the university administration. An optimized lightweight YOLOv8 model is trained to identify three categories: IET stickers (authorized for access), non-IET stickers (unauthorized), and no sticker (denied access). A webcam connected to the Raspberry Pi 4 scans approaching vehicles. Authorized vehicles are allowed when the relevant class is detected, which signals a servo motor to open the gate. Otherwise, access to the gate is denied, and infrared (IR) sensors close the gates. A second set of IR sensors and a servo motor was also added to manage the exit side, preventing unauthorized tailgating. The system's modular design makes it adaptable for different environments, and its use of affordable hardware and open-source tools keeps costs low, which is ideal for smaller institutions or communities. The prototype model is tested and trained on self-collected datasets comprising 506 images.

Keywords: gate access control; YOLOv8; Raspberry Pi 4; artificial intelligence; real-time object recognition; deep learning; vehicle sticker detection; computer vision

check for updates

Academic Editor: Ata Jahangir Moshayedi

Received: 15 September 2025
Revised: 10 October 2025
Accepted: 23 October 2025
Published: 29 October 2025

Citation: Noon, S.K.; Noor, A.H.; Mannan, A.; Arshad, M.; Haider, T.; Abdullah, M. Real-Time Vehicle Sticker Recognition for Smart Gate Control with YOLOv8 and Raspberry Pi 4. *Automation* **2025**, *6*, 63. https://doi.org/10.3390/automation6040063

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Organizations and institutions rely on manual vehicle checks at entry gates across the globe. This involves manual verification of documents by security personnel. The process is slow and cumbersome, often leading to long queues, especially during peak hours [1]. It is also susceptible to human errors, fatigue, and potential security risks, such as unauthorized access due to oversight. In busy settings, these delays cause frustration for drivers and staff while creating bottlenecks that disrupt operations. Commonly used automated gate access control systems employ RFID tags [2]. Magnetic interference and other environmental interferences can make them unreadable [3]. In addition, RFID-based systems come with an additional cost with respect to each readable user-end tag. The low-frequency and passive RFIDs require a close range to work reliably.

In recent years, OCR-based systems have been adopted in several security and surveillance applications [4]. The mechanism, however, requires a license plate recognition (LPR) [5] setup to process the license plate in the image, read its content, and match it with database contents. This approach, however, requires maintaining a database of all registered vehicles [6]. The proposed approach takes on this challenge by developing a sticker-based automatic gate control system. Using real-time object detection [7,8] and sensor technology [9], the system identifies a specific sticker on vehicles to determine entry eligibility. Upon detection, it automatically controls the gate, allowing or denying access.

The main contribution of this paper is to design and implement a real-time automatic vehicle entry system using sticker detection through computer vision and barrier control via servo motors and infrared sensors. To break this down, the specific objectives include the following:

- To develop an optimized lightweight object detection model capable of accurately detecting different types of stickers (e.g., authorized, unauthorized, no sticker) on vehicles in real time.
- To deploy the trained model on a Raspberry Pi 4 kit, enabling real-time processing and decision-making without the need for cloud-based or high-computation infrastructure.
- Design a hardware system using servo motors and infrared sensors to control a barrier gate, responding to sticker detection and vehicle presence.

2. Related Work

Before diving into the development of our automated gate control system, it is important to understand what has already been achieved in the field. The purpose of this literature review is to explore and analyze the foundational works, current technologies, and innovations that helped us understand the research gap. Recent studies have used machine learning- and deep learning-based techniques to control gate access in commercial buildings, gated communities, or parking areas [6]. Traditional systems such as RFID, license plate recognition (LPR), and face detection have made great achievements in presenting solutions for automated gate control, but they are not without limitations. For instance, a fully automated license plate recognition framework was presented that not only localizes and recognizes the license plate but also classifies vehicles without any prior knowledge [10]. The authors employed the YOLOv4 detector to learn independent features from input images and videos. The results highlight the robustness and generalizability of YOLO-based architectures in real-time Automatic License Plate Recognition (ALPR) applications. In another reviewed study [2], a fully automated end-to-end RFID-based toll collection system was developed. RFID-based data authorizes the vehicle after verifying its balance and, upon validation, actuates the servo motor. The motor enables the gate operation via Raspberry Pi control. The system also integrates ultrasonic sensors to ensure smooth entry/exit operations. Table 1 outlines key research efforts in the domain of automated gate systems, focusing on computer vision-based recognition, deep learning applications like YOLO, and other image processing techniques. This motivated us to explore alternative identification methods that require less database management while ensuring more secure and authorized entry.

In a recent work [11], the authors explored the use of deep learning techniques, including OpenCV, in combination with YOLO-based models to develop an ALPR system. The system was then tested at Firat University's entrance. Further, in another published work [7], the authors presented a system design that implements a Raspberry Pi-based facial recognition door lock system using a YOLO-based model. The hardware model was tested for reliability, and its application to real-time control of door locks was carried out.

All prior studies discussed in section led us to develop a sticker-based identification system in simplifying access control mechanisms.

Table 1. Summary of recently published work on vehicle identification-based gate control or other associated applications.

Paper Title	Technique	Advantages	Disadvantages
Saadouli et al. [5]	OCR for license plate recognition, SIFT for car make/model detection, and the Viola–Jones algorithm for face detection	Multi-level authentication enhances security by combining license plate, car model, and face detection.	Limited accuracy (75%) in car make and model recognition. More complex setup due to the fusion of multiple recognition methods
Yaacob et al. [12]	Image processing techniques combined with template matching for license plate recognition	High accuracy in detection (91.58%) and segmentation (91%). Real-time application for automatic campus entry and exit monitoring	The system struggles with complex backgrounds and similar-looking characters. Reduced effectiveness in multi-car scenarios
Shyaa et al. [13]	YOLOv8, compared with Faster R-CNN and SSD for license plate detection	High detection speed and accuracy, improved performance with large datasets, robust in varying conditions	Limited precision on small datasets, computational resource requirements for large datasets
Reda Al-Batat et al. [10]	YOLO-based end-to-end ALPR pipeline for vehicle and license plate detection	Achieved an average recognition accuracy of 90.3% while maintaining an acceptable frames-per-second GPU rate	Limited information about implementing the system for security and gate access control applications
Chandrappa et al. [2]	RFID-based intelligent toll collection using Raspberry Pi, servo motor	Automated toll collection to reduce queuing times	Limited use due to RFID tags, database maintenance to check card entries

3. Materials and Method

Developing an automated secure gate control system via object detection requires a carefully structured software pipeline: from dataset preparation and model training to deployment and real-time execution. This work presents a vehicle sticker-based identification system for automated gate control, implemented on a Raspberry Pi 4 [14] evaluation kit.

The case study is carried out at th NFC Institute of Engineering & Technology, Multan, Pakistan, where the administration issues authorized identification stickers to permitted vehicles (student, visitor, and staff personnel). Currently, security personnel manually check these stickers at the main gate, while vehicles without stickers must undergo clearance before entry. In the proposed system, the automated gate control will open for vehicles with authorized stickers and restrict access (or redirect them for manual entry and checking) for those without them, thereby overcoming the limitations of manual security. The overall methodology involves collecting an image dataset of all probable cars approaching the entry gate, whether they have an authorized sticker, another type of sticker, or no sticker at all. The step-by-step implementation is shown in Figure 1.

The proposed methodology summarized in Figure 1 involves data collection by capturing the images of stickers on cars. By annotating the dataset on Roboflow, this splits our dataset into training, valid, and test data. Then, augmentation techniques were applied to

increase the training dataset. When the dataset completed, we trained it on Google Colab using an optimized lightweight YOLOv8 object detection model [7].

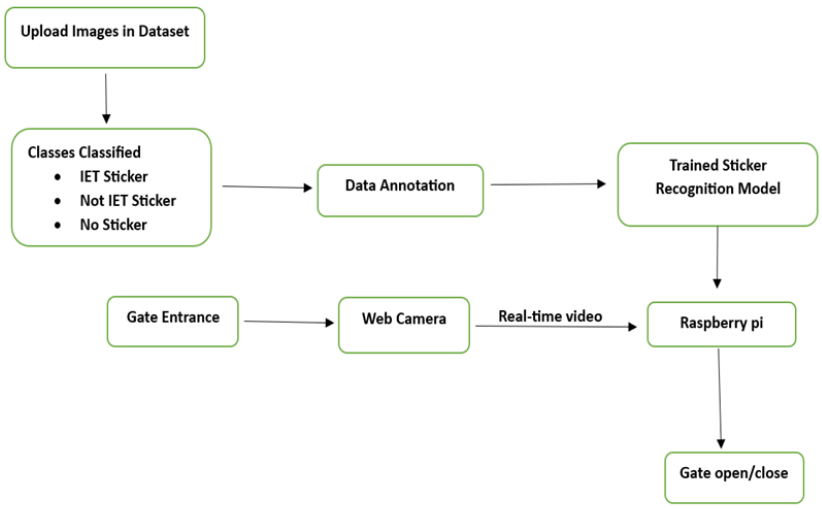


Figure 1. Block diagram of real-time vehicle sticker recognition system for smart gate control using Raspberry Pi 4.

3.1. Optimized Lightweight Detection Model-YOLO v8

The YOLOv8 model was chosen over other types of detectors because YOLO-based architectures, being single-shot detectors, not only provide superior inference speed but also deliver accuracy. So in the context of our gate access control application, inference speeds are a critical requirement, making lightweight YOLO-based models optimal for real-time deployment [9]. YOLOv8 [15] is an advanced object detection model that predicts bounding boxes and class probabilities directly from full images in a single evaluation [16]. The model can be trained easily, and it can be used for image classification, instance segmentation tasks, and object detection.

To enable efficient real-time deployment on Raspberry Pi 4, the YOLOv8-nano model was further optimized by applying 8-bit quantization and pruning redundant parameters. These optimizations resulted in a lightweight model with reduced computational overhead while preserving detection accuracy, making it suitable for low-power gate control applications.

The process of reducing weight and activation precision from 32-bit to 8-bit is known as integer quantization. The process speeds up the inference process in real-time applications, with minimum effects on system accuracy performances. This process comprises applying quantization and de-quantization, and it is described in Equations (1) and (2), where q is a quantized integer (stored as 8-bit), x is the original floating-point value, and the scale factor is mentioned as S. Z is the zero-point (integer that represents the real value 0 in a quantized space) scaling of values from INT-32 to INT-8:

$$q = \text{round}\left(\frac{x}{S}\right) + Z \tag{1}$$

$$\hat{x} = S \cdot (q - Z) \tag{2}$$

The mapping of the real-value range is denoted by x_{min} and x_{max} with respect to the quantized INT-8 range denoted by q_{min} and q_{max}, as shown in Equation (3):

$$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}} \tag{3}$$

Further, to produce the model, the lightweight pruning of parameters is performed to zero-out filters or channel weights that are redundant, as presented in Equation (4):

$$W_i' = \begin{cases} 0, & \text{if } |W_i| < \theta \\ W_i, & \text{otherwise} \end{cases} \quad (4)$$

where W_i' is the weight after pruning, W_i is the parameter showing the original weights. The threshold range is represented by θ in the equation. The YOLOv8 detection head uses a composite loss function, where the localization of the bounding box is improved by selecting a suitable bounding box regression loss. Equation (5) shows the *CIoU* loss function used for our application-specificmodel:

$$L_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad (5)$$

where c is the diagonal length of the smallest enclosing box, v is the aspect ratio's penalty, and α is weight balancing factor.

3.2. Dataset

A reliable deep learning model requires a well-structured and representative dataset [5,17]. The performance of YOLOv8, like any object detection algorithm, is directly dependent on the quality and diversity of its training data [11]. For our system, which classifies vehicle stickers, we created a custom dataset through field collection, a glimpse of which is shown in Figure 2.



Figure 2. Glimpse of self-collected images showing car stickers of different classes.

We manually captured real-world images of stickered vehicles exiting the institutional premises using a standard USB webcam (AR1335 with 13 MP resolution) and a smart phone camera (Samsung A35 and A02s with 13MP resolution; Samsung Electronics Co., Ltd., Suwon-si, Republic of Korea). The captured images have an average resolution of 3120 × 4160 pixels. Each image is stored in JPEG format, with a file size of 2 MB. To ensure robustness and generalization, images were taken under varied environmental conditions:

- Different times of day (morning, noon, and evening);

- Varied lighting conditions, like bright sunlight, overcast, and shadows;
- Multiple angles and distances from the sticker;
- Situations where stickers were partially visible, tilted, or obscured.

After capturing images, we uploaded them on Roboflow [18] to create a dataset for our model. Then, each image was annotated by outlining boxes around the stickers and labeling them according to one of the three following classes:

- Class 0: NFC IET official sticker (authorized);
- Class 1: Non-NFC IET sticker (unauthorized);
- Class 2: No sticker (no visible label).

The distribution of images per class is given in Figure 3. As can be observed, a majority of the samples do not contain any stickers. However, there is also a wide variety of vehicles displaying non-NFC IET stickers along with NFC IET stickers. This diversity of cases supports the practical implementation of the proposed smart gate access control system.

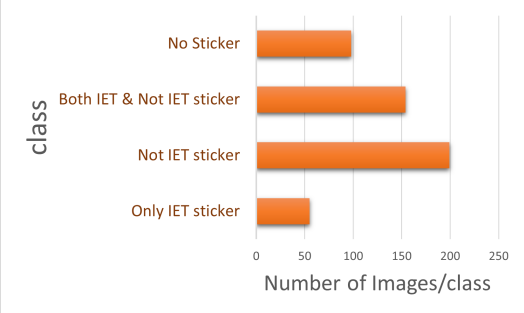


Figure 3. Number of images in each class (in practical cases, class 0 and class 1 coexist).

Once the annotation of 506 total images is complete, the data is split into training, testing, and validation sets in ratios of 70, 10, and 20, respectively. To further generalize the dataset, augmentation was performed on the training set. Rotation, blur, contrast adjustment, flip, zoom, and crop augmentations were applied to make the dataset compatible with real-field scenarios.

3.3. Model Training

After finalizing and exporting the augmented dataset from Roboflow in YOLOv8 format (TXT + JPG in folders named train, valid, and test), we uploaded it to Google Colab, a free cloud-based environment offering GPU acceleration. The input image size for the optimized YOLOv8n model was set to 640 × 640. The model's training was completed using the Ultralytics, Tochvision, and openCV libraries. The PyTorch 1.8 torch.quantization API was used to optimize YOLOv8n for real-time edge deployment. The optimized lightweight YOLOv8 was trained for vehicle sticker recognition to balance its performance between accuracy and efficiency. To adapt the model for Raspberry Pi 4, we applied quantization, which reduced memory usage by approximately 25%, and magnitude-based pruning, which removed nearly 30% of redundant parameters without significant loss in accuracy. The optimized and lightweight model required only 2.2M parameters. Despite the compression, the model maintained a mean average precision (mAP@0.5) of 99% on the training set. The model was trained using an Adam optimizer for 300 epochs with a batch size of 32 and a learning rate of 0.001.

3.4. Experimental Results

The model demonstrates near-perfect performance across all categories. The perfect 100% for "IET-Sticker" and "No-Sticker" classes shows that the model can differentiate these

objects with extreme confidence. Even for the more visually similar "Not-IET-Sticker", accuracy remains extremely high (98.6%). The graph in Figure 4 shows the overall performance of the optimized lightweight YOLOv8 model.

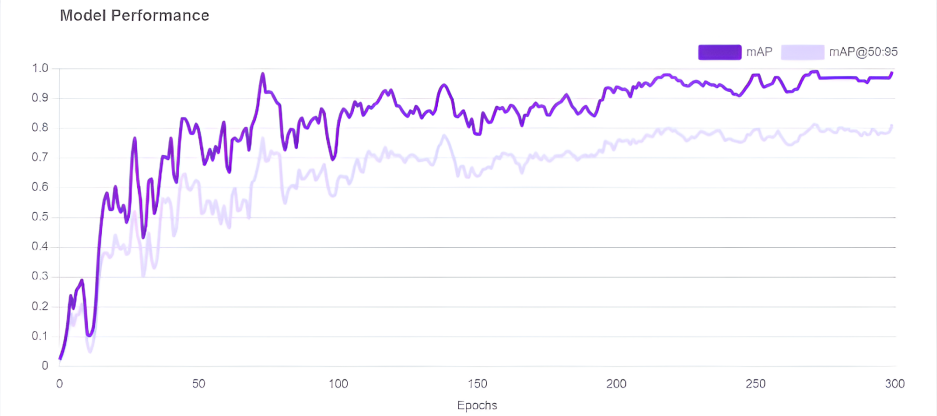


Figure 4. Plot of the optimized lightweight model's training performance.

Table 2 shows a comprehensive summary of the important performance metrics that were needed to carry out analyses before the model's integration in the hardware platform. Higher precision and recall values and lower values of loss indicate that the model can correctly localize and classify the designated class in images or real-time videos. To demonstrate the effectiveness and superior performance of the proposed model, additional YOLO variants were also trained under identical conditions. Since the proposed model is intended for real-time deployment, only lightweight versions of the YOLO architectures were selected for comparison. YOLOv8n was adopted as the baseline model due to its reduced number of parameters and superior accuracy performance compared to other lightweight variants, hence making it well suited for our gate access control dataset, as illustrated in Table 3.

Table 2. Summary of the proposed YOLOv8 model's detection performance following 300 epochs of training.

Metric	Training Value	Validation Value
Precision	0.95–0.97	0.96–0.98
Recall	0.93–0.95	0.94–0.96
mAP@0.5	0.99	0.99
mAP@0.5:0.95	0.88–0.90	0.88–0.90
Box Loss	0.45	0.55
Class Loss	0.25	0.35
Object Loss	0.95	0.97

Table 3. Comparison of proposed optimized YOLOv8 with other detection models in gate access scenarios.

Model	Parameters (M)	Training Time (h)	mAP@50%
YOLOv5n	~1.9	2.5–3.0	0.75
YOLOv6n	~4.3	3.5–4.0	0.72
YOLOv7-tiny	~6.2	4.5–5.0	0.88
YOLOv8n	~3.2	3.0–3.5	0.99
Proposed optimized YOLOv8n	~2.2	2.0–2.3	0.99

3.5. Model Testing

After training, we tested the model using the test set (10% of the total dataset) to validate how well the model generalized to unseen images. The evaluation revealed that the model was capable of identifying stickers with high confidence even under challenging lighting conditions and partial occlusion. The classes with fewer examples, e.g., "No Sticker", performed slightly lower in precision, as can be seen in Figure 5



Figure 5. Model testing on all classes using test set.

The proposed model effectively detects closely placed classes and semi-worn-out stickers, as illustrated in Figures 5 and 6. However, since the training dataset did not include any severely discolored or damaged sticker images, it was not possible to fully evaluate the model's robustness under such challenging conditions using the test data. Addressing these scenarios is part of our future work, where the dataset will be expanded to include more difficult and realistic cases to assess the model's performance under real-world, worn-out conditions.



Figure 6. Successful detection by the model despite sticker wear and discoloration.

4. Hardware Implementation

To ensure reliability and real-time performance, the hardware implementation of the automated gate control system was implemented using Raspberry Pi-4. Raspberry Pi 4 serves as the central processor, and it is used to handle the process of localizing and classifying the type of sticker on the vehicle and further execute the gate control mechanism. A webcam attached to it will process real-time videos, and this enables the servo motor to open the gate in authorized vehicle cases. For the given image resolution, the optimized YOLOv8n model runs on Raspberry Pi 4 with 4 GB RAM and a Quad-Core Cortex-A72 processor, achieving about 3–5 FPS. The frame rate can be improved by reducing the image's size, although this results in a corresponding decrease in accuracy. The key connected components are IR sensor modules, servo motors [19], LEDs with the GPIO pins of Raspberry Pi [20], and a webcam using the USB port of Raspberry Pi to capture real-time videos of gate entrances. Each hardware component played a specific role, collectively enabling the automatic operation of the entry gate. The overall schematic of the hardware model is shown in Figure 7.

A USB webcam was used as the visual input device to continuously stream live videos to the Raspberry Pi 4. The camera feeds real-time frames to the model for object detection. Servo motors were employed to physically control the opening and closing of the entry barrier [21]. The SG90 servo is compact and lightweight, and it provides precise angular movements based on PWM signals from the Raspberry Pi's GPIO pins [20]. This is visualized in Figure 7, where one motor is responsible for lifting the barrier upon sticker verification, while the second motor operates based on vehicle exit detection.

We used three IR sensors to detect the presence of a vehicle at critical points—before the gate, under the barrier, and past the gate exit. These IR sensors function as the system's "eyes" for authorization and help automate the open–close control of the barrier. IR sensors provide fast digital outputs, are inexpensive, and operate reliably in outdoor conditions [21].

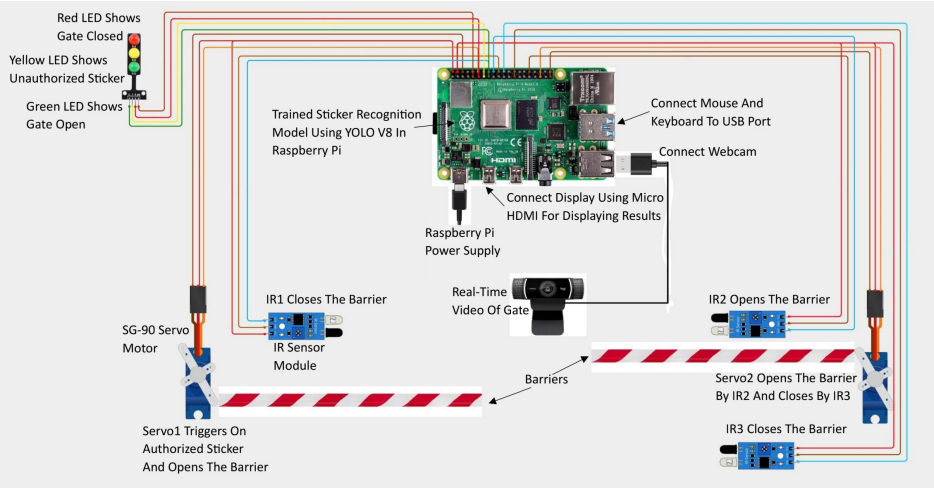


Figure 7. Schematic diagram of the hardware model used to enable gate opening and closing via authorized sticker recognition.

The details and specifications of the hardware components are mentioned in Table 4.

Table 4. Hardware components and specifications.

Component	Specification	Description
Infrared Sensor	3.3 V–5 V	Provides digital output (HIGH/LOW) for obstacle detection.
Micro Servo Motor-SG90	4.8 V–6 V	Rotation: 0°–180°; torque: ~2.5 kg-cm. Used to control barrier movement.
LED (Green) LED (Red)	Indicator Indicator	Sticker verified; barrier opening. Barrier closed/Unauthorized or failed detection.
LED (Green)	Indicator	Sticker verified; barrier opening.
LED (Yellow)	Indicator	Detection in progress.
Power Supply	5 V, 2.5 A Adapter	Provides stable power to Raspberry Pi 4 and its peripherals.

5. Working Model Prototype

The trained optimized lightweight model file named best.pt is saved in the Raspberry Pi 4 for deployment. Subsequently, the required python libraries are installed; then, a code sequentially loads the the trained model, accesses the webcam, interprets the detection results, controls hardware components, and loops the process continuously. The code is saved as project.py in the project folder. The Raspberry Pi uses its connected webcam to continuously capture live video frames, which are then passed through the trained YOLOv8 model for real-time detection. If the model identifies an IET sticker (class 0), LED3 (green) turns on, and Servo1 opens the barrier. Once the vehicle crosses IR sensor 1, the barrier automatically closes. If the model detects any other sticker (class 1 or class 2), LED2 (yellow) starts blinking continuously, indicating restricted access, while LED1 remains on when no sticker is detected. Additionally, Servo2 is controlled through IR sensor 2 and IR sensor 3: It opens when IR2 detects a vehicle and remains open until IR3 confirms passage, after which it closes. The whole schematic can be understood using the schematic diagram shown in Figure 7. To prevent false closure cases, IR3 is placed ahead of IR2. There may be a case where a slow-moving vehicle passes and crosses IR2 (i.e., no vehicle is detected) but still does not trigger IR3. The barrier will not close until IR3 is triggered. Moreover, a 2–3 s delay is added before sending the close signal to the servo motor. The overall system is shown in Figure 7. The proposed smart gate control mechanism avoids false gate closures, and if vehicles are moving continuously, the system keeps the gate open until all sensors read “clear”.

To better understand the decision-making logic of the system, we created a flowchart as shown in Figure 8 that represents the operational flow from image capture to servo control. This visual model helps break down the conditional checks and actions into a step-by-step logical sequence.

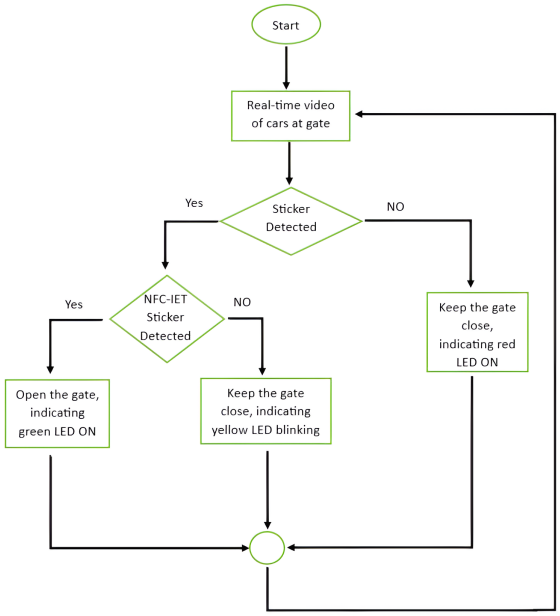


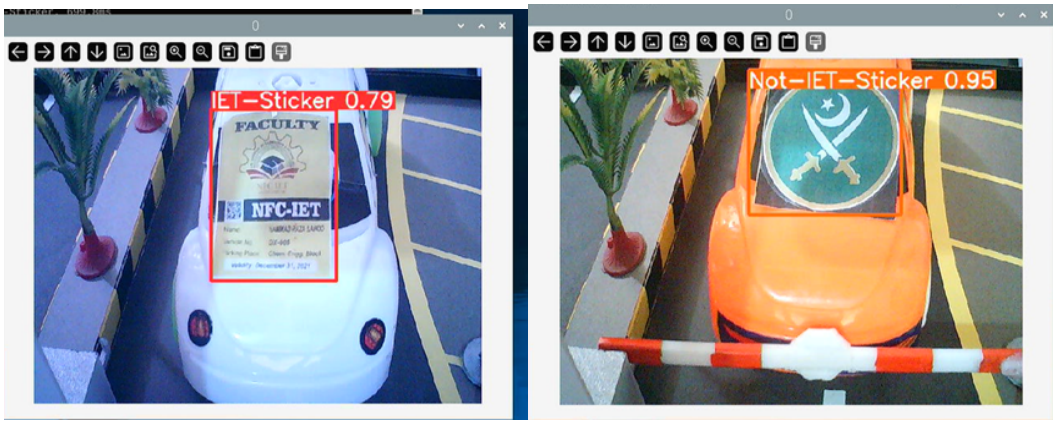
Figure 8. Logical decision-making flow of the model.

To test the real-time performance of the model, a prototype was built in which 15 model toy cars with varying classes of stickers were used to check the control of gate opening and closing. The prototype model is shown in Figure 9.



Figure 9. Prototype model of automated vehicle sticker recognition-based gate control.

The detection results of the model, tested for its ability to detect and recognize multiple authorized unauthorized cars from varying distances, are depicted in Figure 10.



(**a**) (**b**)

Figure 10. Detection results from Raspberry Pi 4 prototype for automated gate control via vehicle sticker recognition. (**a**) Detection of authorized vehicle with sticker; (**b**) unauthorized vehicle without sticker.

6. Limitations of This Study

The real-time performance of the system may be compromised during rush hours. This is because processing delays can occur due to the limited computational power of Raspberry Pi, resulting in slower inference. Also, the model's performance can degrade under poor lightning conditions, as the training images are captured in bright lighting conditions. Furthermore, the system authorizes the vehicle solely based on sticker authentication, irrespective of who is driving the car.

7. Conclusions and Future Work

This study implemented an automated real-time vehicle sticker recognition gate control mechanism using a lightweight optimized model. The model deployed for real-time operations performed well on training and test data under varying conditions. The YOLOv8 model was further optimized and pruned to make its performance suitable for real-time applications. The setup integrates a webcam, IR sensors, servo motors, and LED indicators to identify authorized vehicles with IET stickers and control the gate accordingly. When an authorized sticker is detected, the gate opens, and sensors ensure tha it closes safely after the vehicle passes. The system performed reliably across different lighting conditions. The work shows how machine learning and IoT can tackle real-world issues like campus security, traffic flow, and automation, making access control smarter and more efficient. Although the proposed system demonstrates reliable performance in various real-world scenarios, there is always room for improvement, so, in the future, we plan to expand the dataset to improve accuracy and generalization. To further enhance security, additional features such as facial recognition and vehicle make/model identification can be incorporated. Facial embeddings can be used to match authorized personnel stored in the access database for enhanced authentication. Moreover, the development of a user-friendly application for the remote monitoring of gate access logs and secure data storage is also under consideration. This can be enabled by linking the Raspberry Pi to a cloud or web server via REST API or MQTT for real-time remote access and alerts.

Author Contributions: Conceptualization, S.K.N. and A.M.; methodology, S.K.N. and A.M.; software, A.H.N., M.A. (Miqdam Arshad), and T.H.; validation, S.K.N., A.H.N., and M.A. (Muhammad

Abdullah); formal analysis, A.H.N., M.A. (Miqdam Arshad), T.H., and M.A. (Muhammad Abdullah); investigation, A.H.N.; resources, S.K.N. and A.M.; data curation, A.H.N., M.A. (Miqdam Arshad), T.H., and M.A. (Muhammad Abdullah); writing—original draft preparation, S.K.N., A.H.N., and A.M.; writing—review and editing, A.M.; supervision, S.K.N.; project administration, A.M. All authors have read and agreed to the published version of this manuscript.

Funding: This research received no external funding.

Data Availability Statement: The dataset will be shared upon request.

Acknowledgments: The authors acknowledge the support extended by the university's administration, faculty students, and staff in the acquisition of the dataset.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Elechi, P.; Ahiakwo, C.O.; Shir, S.T. Design and Implementation of an Automated Security Gate System Using Global System for Mobile Communication Network. *J. Netw. Comput. Appl.* **2021**, *7*, 1–10.
2. Chandrappa, S.; Guruprasad, M.S.; Kumar, H.N.N.; Raju, K.; Kumar, D.K.S. An IoT-Based Automotive and Intelligent Toll Gate Using RFID. *SN Comput. Sci.* **2023**, *4*, 154. [CrossRef]
3. Alvarez-Narciandi, G.; Motroni, A.; Rodriguez Pino, M.; Buffi, A.; Nepa, P. A UHF-RFID Gate Control System Based on a Recurrent Neural Network. *IEEE Antennas Wirel. Propag. Lett.* **2019**, *18*, 2330–2334. [CrossRef]
4. Peng, J. Onstruction and Security Performance Analysis of an Anti-Attack Optical Character Recognition (OCR) System. In Proceedings of the 2024 International Conference on Information Technology, Communication Ecosystem and Management (ITCEM), Bangkok, Thailand, 20–22 December 2024; pp. 140–144.
5. Saadouli, G.; Elburdani, M.I.; Al-Qatouni, R.M.; Kunhoth, S.; Al-Maadeed, S. Automatic and Secure Electronic Gate System Using Fusion of License Plate, Car Make Recognition and Face Detection. In Proceedings of the 2020 IEEE International Conference on Informatics, IoT, and Enabling Technologies (ICIoT), Doha, Qatar, 2–5 February 2020; pp. 79–84.
6. Revanth, Y.; Namitha, K. License Plate Recognition-Based Automatic Gate Opening System. In Proceedings of the 2024 IEEE International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS), Bangalore, India, 28–29 June 2024; pp. 1–6.
7. Elnozahy, S.S.F.A.; Pari, S.C.; Liang, L.C. Raspberry Pi-Based Face Recognition Door Lock System. *IoT* **2025**, *6*, 31. [CrossRef]
8. Chen, Z.; Yang, J.; Chen, L.; Li, F.; Feng, Z.; Jia, L.; Li, P. RailVoxelDet: A Lightweight 3D Object Detection Method for Railway Transportation Driven by On-Board LiDAR Data. *IEEE Internet Things J.* **2025**, *12*, 37175–37189. [CrossRef]
9. Al Amin, R.; Hasan, M.; Wiese, V.; Obermaisser, R. FPGA-Based Real-Time Object Detection and Classification System Using YOLO for Edge Computing. *IEEE Access* **2024**, *12*, 73268–73278. [CrossRef]
10. Al-Batat, R.; Angelopoulou, A.; Premkumar, S.; Hemanth, J.; Kapetanios, E. An End-to-End Automated License Plate Recognition System Using YOLO-Based Vehicle and License Plate Detection with Vehicle Classification. *Sensors* **2022**, *22*, 9477. [CrossRef] [PubMed]
11. Mustafa, T.; Karabatak, M. Real-Time Car Model and Plate Detection System by Using Deep Learning Architectures. *IEEE Access* **2024**, *12*, 107616–107630. [CrossRef]
12. Yaacob, N.L.; Alkahtani, A.A.; Noman, F.M.; Zuhdi, A.W.M.; Habeeb, D. License Plate Recognition for Campus Auto-Gate System. *Indones. J. Electr. Eng. Comput. Sci.* **2021**, *21*, 128–136. [CrossRef]
13. Shyaa, T.A.; Hashim, A.A. Superior Use of YOLOv8 to Enhance Car License Plates Detection Speed and Accuracy. *Rev. D'Intell. Artif.* **2024**, *38*, 1. [CrossRef]
14. Ahmad, B.; Noon, S.K.; Ahmad, T.; Mannan, A.; Khan, N.I.; Ismail, M.; Awan, T. Efficient Real-Time Detection of Plant Leaf Diseases Using YOLOv8 and Raspberry Pi. *VFAST Trans. Softw. Eng.* **2024**, *12*, 250–259. [CrossRef]
15. Liu, Y.; Shen, S. Vehicle Detection and Tracking Based on Improved YOLOv8. *IEEE Access* **2025**, *13*, 24793–24803. [CrossRef]
16. Arora, S.; Mittal, R.; Arora, D.; Shrivastava, A.K. A Robust Approach for Licence Plate Detection Using Deep Learning. *Intel. Artif.* **2024**, *27*, 129–141. [CrossRef]
17. Bukola, A.C.; Owolawi, P.A.; Du, C.; Van Wyk, E. A Systematic Review and Comparative Analysis Approach to Boom Gate Access Using Plate Number Recognition. *Computers* **2024**, *13*, 286. [CrossRef]
18. Noon, S.K.; Amjad, M.; Qureshi, M.A.; Mannan, A.; Awan, T. An Improved Detection Method for Crop & Fruit Leaf Disease Under Real-Field Conditions. *AgriEngineering* **2024**, *6*, 344–360. [CrossRef]
19. Karim, M.Z.B.A.; Thamrin, N.M. Servo Motor Controller Using PID and Graphical User Interface on Raspberry Pi for Robotic Arm. *J. Phys. Conf. Ser.* **2022**, *2319*, 012015. [CrossRef]

20. Küçükdermenci, S. Raspberry Pi-Based Real-Time Parking Monitoring with Mobile App Integration. In Proceedings of the 5th International Conference on Engineering and Applied Natural Sciences, Konya, Turkey, 10–12 July 2023; pp. 1458–1464.
21. Ingale, K.; Tekade, O.; Thakare, P.; Wankhade, A.; Wankhade, P. An Advanced Surveillance Motorized Car for Real-Time Inspection with Sensing Capabilities. In Proceedings of the International Conference on Robotics, Control, Automation and Artificial Intelligence, Manipal, India, 12–14 October; pp. 263–279.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.